

자료구조 실습 보고서

[제 04 주] 성능 측정

제출일 : 2018 년 3 월 25 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘, 자료구조

이 프로그램은 입출력을 담당하는 AppView 클래스,
전체적인 제어를 담당하는 AppController 클래스,
데이터를 생성하여 성능을 측정하는 PerformanceMeasurement 클래스,
비정렬상태의 가방을 구현하는 UnsortedArrayBag 클래스,
값을 가지는 동전을 구현하는 Coin 클래스,
측정 결과를 담는 TestResult 클래스,
프로그램 메시지 출력을 용이하게 하기 위해 사용되는 MessageID enum 으로 구성된다.

- public void testUnsortedArrayBag()

최댓값을 계산하는데 걸리는 시간과, 가방에 새로운 원소를 삽입할 때 걸리는 시간을 계산한다.
처음 데이터의 크기를 정해 놓고, 크기를 일정하게 증가시키며 그 때마다 걸리는 시간을 측정한다.
테스트가 새로 시행될 때마다 정해진 데이터의 크기에 맞게 가방의 크기가 결정된다.
시간 측정은 System.nanoTime() 메소드를 통하여 처음 시간과 나중 시간의 차이를 계산함으로써 이루어진다.
시간 측정이 끝나면, TestResult 형의 배열들에 정보를 담는다.

b. 함수 설명서

■ UnsortedArrayBag

- public UnsortedArrayBag(int givenCapacity) : 가방의 크기를 설정. 인자를 넣지 않을 시, 100 으로 설정 (생성자)
- public int size() : 가방 크기 반환 (Getter)
- public boolean isEmpty() : 가방 비어있는지 여부 반환
- public boolean isFull() : 가방이 다 찼는지 여부 반환
- public E elementAt(int order_p) : 인자로 받은 인덱스에 있는 원소 반환 (Getter)
- public boolean doesContain(E element_p) : 인자로 받은 원소와 같은 값의 원소가 있는지 여부 반환
- public int frequencyOf(E element_p) : 인자로 받은 원소와 같은 값의 원소가 몇 개 있는지 반환
- public boolean add(E element_p) : 인자로 받은 원소를 가방에 추가

- `public boolean remove(E element_p)` : 인자로 받은 원소와 같은 값을 갖는 원소 1 개 제거
- `public void clear()` : 가방 비움

■ AppController

- `public void run()` : 프로그램 실행

■ AppView

- `public void outputMessage(String aMessageString)` : 문자열 출력
- `public void outputResult(int aTestSize, long aTestInsertTime, long aTestFindMaxTime)` : 결과 출력 (테스트한 데이터의 크기, 삽입 시 걸린 시간, 최댓값 검색 시간)

■ Coin

- `public Coin(int givenValue)` : 코인에 값 설정. (생성자)
- `public int value()` : 코인 값 반환 (Getter)
- `public void setValue(int newValue)` : 코인 값 설정 (Setter)
- `public boolean equals(Object coin_p)` : 코인 값 비교 후 같은 지 반환

■ PerformanceMeasurement

- `public PerformanceMeasurement(int givenNumberOfTests, int givenFirstSize, int givenSizeIncrement)` : 테스트 시행 횟수, 처음 시행 시 데이터 크기, 데이터 크기 증가량 설정. 인자를 넣지 않을 시, 5, 10000, 10000 으로 설정 (생성자)
- `public int numberOfTests()` : 테스트 시행 횟수 반환 (Getter)
- `public TestResult[] testResults()` : 테스트 결과 반환 (Getter)
- `public int maxDataSize()` : 데이터 최대 크기 반환
- `public void generateData()` : 난수 데이터 생성
- `public void testUnsortedArrayBag()` : UnsortedArrayBag 에 대한 성능 테스트를 수행한다.

■ TestResult

- `public TestResult(int aTestSize, long aTestInsertTime, long aTestFindMaxTime)` : 테스트 시행 횟수, 삽입 소요 시간, 최댓값 검색 시간 설정 (생성자)
- `public int testSize()` : 테스트 시행 횟수 반환 (Getter)
- `public long testInsertTime()` : 삽입 소요 시간 반환 (Getter)
- `public long testFindMaxTime()` : 최댓값 검색 시간 반환 (Getter)
- `public void setTestSize(int aTestSize)` : 테스트 시행 횟수 설정 (Setter)
- `public void setTestInsertTime(int aTestInsertTime)` : 삽입 소요 시간 설정 (Setter)
- `public void setTestFindMaxTime(int aTestFindMaxTime)` : 최댓값 검색 시간 설정 (Setter)

c. 종합 설명서

이번 과제는 2 주차 과제에서 설계하였던 Unsorted ArrayBag 자료 구조에서, 데이터의 크기에 따라서 삽입과 최댓값 검색에 걸리는 시간의 변화 추이를 보인다. 프로그램을 실행하면, 데이터의 크기를 증가하며 여러 번 테스트를 수행하며 결과를 출력한다.

2. 프로그램 장단점 분석

장점 : x

단점 : 시간이 너무 오래 걸리면, 출력 결과가 깔끔하게 정렬되지 않는다.

3. 실행 결과 분석

a. 입력과 출력

```
<<< 실행 성능 차이 알아보기 >>>
[Unsorted Array List]
크기 :   10000  삽입하기 :    593323  최대값찾기 :   42039815
크기 :   20000  삽입하기 :   1332514  최대값찾기 :  198409671
크기 :   30000  삽입하기 :   2326293  최대값찾기 :  415160461
크기 :   40000  삽입하기 :   2800957  최대값찾기 :  692046209
크기 :   50000  삽입하기 :   2199383  최대값찾기 : 1278905876
<<< 성능 측정을 종료합니다. >>> |
```

b. 결과분석

크기가 증가할수록 전체적으로는 증가하는 추이를 보였으나, 40000 ~ 50000 에서 삽입 소요 시간의 변화를 보면 반드시 그런 것 만은 아니라는 결과를 도출할 수 있었다.

4. 소스코드

```
AppController.java
// 총체적인 제어 담당

public class AppController {

    private AppView _appView;
    private PerformanceMeasurement _pm;

    // 생성자
    public AppController() {
        this._appView = new AppView();
    }

    // 실행
```

```

public void run() {

    this._pm = new PerformanceMeasurement();

    this.showMessage(MessageID.Notice_StartProgram);
    this._pm.generateData();

    this.showMessage(MessageID.Notice_UnsortedArrayStart);           // 자료구조
    이름 명시

    this._pm.testUnsortedArrayBag();

    this.showTestResults();

    this.showMessage(MessageID.Notice_EndProgram);

}

// 상황에 맞는 메시지 출력
private void showMessage(MessageID aMessageID) {
    switch(aMessageID) {
        case Notice_StartProgram:
            this._appView.outputMessage("<<< 실행 성능 차이 알아보기 >>> \n");
            break;

        case Notice_EndProgram:
            this._appView.outputMessage("<<< 성능 측정을 종료합니다. >>> \n");
            break;

        case Notice_UnsortedArrayStart:
            this._appView.outputMessage("[Unsorted Array List] \n");
            break;

        case Error_WrongMenu:
            this._appView.outputMessage("<<< Error : 잘못된 메뉴입니다. >>> \n");
            break;

        default:
            break;
    }
}

// 결과 출력
private void showTestResults() {
    TestResult testResults[] = this._pm.testResults();
    for(int index = 0; index < this._pm.numberOfTests(); index++) {
        this._appView.outputResult(testResults[index].testSize(),
testResults[index].testInsertTime(),
testResults[index].testFindMaxTime());
    }
}
}

```

AppView.java

// 출력

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 문자열 출력
    public void outputMessage(String aMessageString) {
        System.out.print(aMessageString);
    }

    // 결과 출력
    public void outputResult(int aTestSize, long aTestInsertTime, long
aTestFindMaxTime) {
        System.out.printf("크기 : %7d", aTestSize);
        System.out.printf(" 삽입하기 : %10d", aTestInsertTime);
        System.out.printf(" 최대값찾기 : %10d", aTestFindMaxTime);
        System.out.println();
    }

}

```

PerformanceMeasurement.java

```

// 성능 측정 및 데이터 생성
// 처음 데이터 크기를 정해놓고, 크기를 일정하게 증가시키며 특정 횟수 시행

import java.util.Random;

public class PerformanceMeasurement {

    public static final int DEFAULT_NUMBER_OF_TESTS = 5;
    public static final int DEFAULT_FIRST_TEST_SIZE = 10000;
    public static final int DEFAULT_SIZE_INCREMENT = 10000;

    private int _numberOfTests;           // 시행 횟수
    private int _firstTestSize;           // 처음 시행 시 데이터 크기
    private int _sizeIncrement;           // 시행 시마다 크기 증가량

    private int[] _data;                   // 난수를 저장할 데이터
    private TestResult[] _testResults;     // 시행 결과

    // 생성자
    public PerformanceMeasurement() {
        this._numberOfTests = PerformanceMeasurement.DEFAULT_NUMBER_OF_TESTS;
        this._firstTestSize = PerformanceMeasurement.DEFAULT_FIRST_TEST_SIZE;
        this._sizeIncrement = PerformanceMeasurement.DEFAULT_SIZE_INCREMENT;
    }

```

```

        // 필요한 최대 크기만큼 미리 데이터 틀 생성
        this._data = new int[this.maxDataSize()];

        // 시행 횟수만큼 결과 생성
        this._testResults = new
        TestResult[PerformanceMeasurement.DEFAULT_NUMBER_OF_TESTS];
    }
    public PerformanceMeasurement(int givenNumberOfTests, int givenFirstTestSize, int
    givenSizeIncrement) {
        this._numberOfTests = givenNumberOfTests;
        this._firstTestSize = givenFirstTestSize;
        this._sizeIncrement = givenSizeIncrement;

        this._data = new int[this.maxDataSize()];
        this._testResults = new TestResult[givenNumberOfTests];
    }

    // Getter
    public int numberOfTests() {
        return this._numberOfTests;
    }
    public TestResult[] testResults() {
        return this._testResults;
    }

    // 실험 데이터 최대 크기
    public int maxDataSize() {
        return this._firstTestSize + this._sizeIncrement * (this._numberOfTests - 1);
    }

    // 실험 데이터 생성
    public void generateData() {
        Random random = new Random();

        for(int i = 0; i < this.maxDataSize(); i++) {
            this._data[i] = random.nextInt(this.maxDataSize());
        }
    }

    // UnsortedArrayBag 테스트 수행
    public void testUnsortedArrayBag() {
        UnsortedArrayBag<Coin> bag;
        int maxCoin; // 코인 최댓값

        int testSize; // 데이터 크기

        long timeForAdd, timeForMax; // 걸린 시간
        long start, stop; // 시작시간, 정지시간

        testSize = this._firstTestSize;

        for(int testCount = 0; testCount < this._numberOfTests; testCount++) {

            bag = new UnsortedArrayBag<Coin>(testSize); // 데이터 크기 :

```

가방 크기

```
timeForAdd = 0;
timeForMax = 0;

for(int testDataCount = 0; testDataCount < testSize; testDataCount++) {
    Coin coin = new Coin(this._data[testDataCount]);    // 코인의
```

값에 난수 지정

```
    // 삽입 시간 측정
    start = System.nanoTime();
    bag.add(coin);
    stop = System.nanoTime();
    timeForAdd += (stop - start);

    // 최댓값 계산 시간 측정
    start = System.nanoTime();
    maxCoin = this.maxCoinValue(bag);
    stop = System.nanoTime();
    timeForMax += (stop - start);
}
```

```
    this._testResults[testCount] = new TResult(testSize, timeForAdd,
timeForMax);
    testSize += this._sizeIncrement;
}
```

```
}
```

```
private int maxCoinValue(UnsortedArrayBag<Coin> aCoinBag) {
    int maxValue = 0;
    for(int i = 0; i < aCoinBag.size(); i++) {
        if(maxValue < aCoinBag.elementAt(i).value()) {
            maxValue = aCoinBag.elementAt(i).value();
        }
    }
    return maxValue;
}
```

```
}
```

TestResult.java

// 결과 정보

```
public class TResult {

    private int _testSize;
    private long _testInsertTime;
    private long _testFindMaxTime;

    // 생성자
    public TResult() {

    }
    public TResult(int aTestSize, long aTestInsertTime, long aTestFindMaxTime) {
        this._testSize = aTestSize;
        this._testInsertTime = aTestInsertTime;
        this._testFindMaxTime = aTestFindMaxTime;
    }
}
```



```

    }

    // Getter
    public int testSize() {
        return this._testSize;
    }
    public long testInsertTime() {
        return this._testInsertTime;
    }
    public long testFindMaxTime() {
        return this._testFindMaxTime;
    }

    // Setter
    public void setTestSize(int aTestSize) {
        this._testSize = aTestSize;
    }
    public void setTestInsertTime(int aTestInsertTime) {
        this._testInsertTime = aTestInsertTime;
    }
    public void setTestFindMaxTime(int aTestFindMaxTime) {
        this._testFindMaxTime = aTestFindMaxTime;
    }
}

```

UnsortedArrayBag.java

// 가방에 대한 정보

```

public class UnsortedArrayBag<E> {

    // 상수
    private static final int DEFAULT_CAPACITY = 100;

    // 인스턴스 변수

    // 가방 최대 용량
    private int _capacity;

    // 현재 가방 용량
    private int _size;

    // 가방은 E의 배열로 구성됨
    private E[] _elements;

    // 생성자
    public UnsortedArrayBag() {
        this._capacity = UnsortedArrayBag.DEFAULT_CAPACITY;
        this._elements = (E[])new Object[this._capacity];
        this._size = 0;
    }

    public UnsortedArrayBag(int givenCapacity) {

```

```

        this._capacity = givenCapacity;
        this._elements = (E[])new Object[this._capacity];
        this._size = 0;
    }

    // 현재 크기 반환 : getter
    public int size() {
        return this._size;
    }

    // 가방 비어있는지 여부 반환
    public boolean isEmpty() {
        return (this._size == 0);
    }

    // 가방 다 찼는지 여부 반환
    public boolean isFull() {
        return (this._size == _capacity);
    }

    // 주어진 order에 있는 원소 반환
    public E elementAt(int order_p) {
        if((0 <= order_p) && (order_p < this.size())) {
            return this._elements[order_p];
        }
        else {
            return null;
        }
    }

    // 주어진 element과 같은 값의 element이 있는지 여부 반환
    public boolean doesContain(E element_p) {

        boolean found = false;

        for(int index = 0; index < this._size; index++) {
            if(element_p.equals(this._elements[index])) {
                found = true;
                break;
            }
        }
        return found;
    }

    // 주어진 element과 같은 값의 element이 몇 개 있는지 반환
    public int frequencyOf(E element_p) {

        int frequencyCount = 0;

        for(int index = 0; index < this._size; index++) {
            if(element_p.equals(this._elements[index])) {
                frequencyCount++;
            }
        }
        return frequencyCount;
    }

```

```

// 가방에 element 추가
public boolean add(E element_p) {

    if(this.isFull()) {
        return false;
    }
    else {
        this._elements[this._size] = element_p;
        this._size++;
        return true;
    }
}

// 가방에서 주어진 값의 element 삭제
public boolean remove(E element_p) {

    boolean found = false;

    if(this.isEmpty()) {
        return found;
    }

    // 삭제 시
    else {

        // 가방에 element 1개뿐일 때, 그냥 비우기
        if(this._size == 1) {
            found = this._elements[0].equals(element_p);
            this._elements[0] = null;
            this._size--;
        }

        // 가방에 element가 2개 이상일 때
        for(int index = 0; index < this._size - 1 && !found; index++) {

            found = this._elements[index].equals(element_p);

            // 마지막 원소는 순회되지 않으므로 따로 확인
            found = this._elements[this._size - 1].equals(element_p);

            // 같은 것을 찾으면, 그 인덱스부터 배열 요소 번호들을 1씩 앞으로
            // 당기고 마지막 원소는 null로 초기화
            if(found == true) {
                for(int j = index; j < this._size - 1; j++) {
                    this._elements[j] = this._elements[j + 1];
                }
                this._elements[this._size - 1] = null;
                this._size--;
            }
        }

        return found;
    }
}

// 가방 초기화
public void clear() {
    this._size = 0;
}

```

```
}
```

Coin.java

```
// 동전에 대한 정보
```

```
public class Coin {
```

```
    // 인스턴스 변수
```

```
    // 금액
```

```
    private int _value;
```

```
    // 생성자
```

```
    public Coin() {    }
```

```
    public Coin(int givenValue) {  
        this._value = givenValue;  
    }
```

```
    // 코인 금액 반환 : getter
```

```
    public int value() {  
        return this._value;  
    }
```

```
    // 코인 금액 설정 : setter
```

```
    public void setValue(int newValue) {  
        this._value = newValue;  
    }
```

```
    // 코인 금액 비교
```

```
    @Override
```

```
    public boolean equals(Object coin_p) {
```

```
        // 인자로 받은 값이 Coin형이 아닐 경우
```

```
        if(coin_p.getClass() != Coin.class) {  
            return false;  
        }
```

```
        // Coin형을 받았을 경우 금액 비교
```

```
        else {  
            return (this.value() == ((Coin)coin_p).value());  
        }
```

```
    }
```

```
}
```

MessageID.java

```
public enum MessageID {
```

```
    // Message IDs for Notices
```

```
    Notice_StartProgram,
```

```
    Notice_EndProgram,
```

```
    Notice_UnsortedArrayStart,
```

```
// Message IDs for Errors  
Error_WrongMenu
```

```
}
```

DS1_04_201702081_최재범

```
// main
```

```
public class DS1_04_201702081_최재범 {  
  
    public static void main(String[] args) {  
  
        ApplicationController appController = new ApplicationController();  
        appController.run();  
  
    }  
  
}
```